

GUÍA DE ARQUITECTURA · DEVOPS EN AWS

De un servidor a tres: API con balanceo de carga, HTTPS y WebSockets en AWS

Qué construimos, por qué, cuánto cuesta, cómo operarlo y cómo reutilizarlo con cualquier backend. Escrita para que la entienda tanto quien empieza como quien ya trabaja en infraestructura.

Terraform · Ansible · EC2 · ALB · NLB · RDS · ElastiCache Redis · Docker · NestJS · Socket.IO

En palabras simples — explicación sin tecnicismos

Detalle técnico — comandos, valores y configuración

Palabra clave — conceptos que conviene memorizar (ver glosario)

Proyecto **infra-sysbar** · rama `multi-servers` · Noviembre 2026 · Precios de referencia: us-east-1, verificados en noviembre de 2026

Contenido

1. El objetivo: qué se esperaba lograr
2. La arquitectura y por qué es un estándar en DevOps
3. Servicios de AWS utilizados y su razón de ser
4. Terraform y Ansible en pocas palabras
5. Lo que necesitas instalado y configurado
6. Lo que hicimos, paso a paso
7. Cómo funciona el ALB (y cuántas instancias soporta)
8. El NLB: para qué sirve y por qué lo colocamos
9. Costos: por minuto, por día, por mes
10. ¿Solo funciona con SysRanchito / NestJS?
11. Qué hacer en caso de fallas
12. Operación del día a día
13. Seguridad
14. Mejoras recomendadas
15. Glosario de palabras clave

1 El objetivo: qué se esperaba lograr

El punto de partida y la meta

En palabras simples

La aplicación vivía en **una sola computadora** en la nube. Si esa computadora se caía, se saturaba o había que actualizarla, el restaurante se quedaba sin sistema. La meta era repartirla en **tres computadoras** con un "recepcionista" que reparta a los usuarios, manteniendo la misma dirección web segura (HTTPS) y la comunicación en tiempo real (WebSockets).

Antes

- 1 instancia EC2 con Nginx + Certbot + la API.
- Terraform y Ansible pensados para **un solo servidor**.
- Puertos 22, 80, 443 y 3000 abiertos a todo internet.
- Un error en el inventario de Ansible (`[webserver]` vs `webservers`) impedía que el playbook corriera.
- Despliegue por GitHub Actions a un único servidor.

Requisitos del objetivo

- Un **Application Load Balancer (ALB)** que reparta el tráfico.
- **3 instancias EC2** (tamaño mínimo viable: es entorno de prueba).
- **Una sola IP elástica** para el dominio `sysbarrestaurant.duckdns.org`.
- **HTTPS** y **WebSockets** funcionando entre servidores.
- Todo como **Infraestructura como Código (IaC)**: Terraform + Ansible.
- Región **us-east-1**; base de datos sin cambios de tamaño.

Resultado alcanzado y verificado

Prueba	Resultado	Qué demuestra
6 peticiones a `/api/health`	**Respondieron ec2-1, ec2-2 y ec2-3**	El balanceador reparte la carga
Logs de los 3 contenedores	**"Socket.IO conectado a Redis" ×3**	Los servidores comparten eventos
Script con 9 clientes WebSocket	**9/9 recibieron el evento**	Tiempo real entre instancias
Ansible `deploy.yml`	**failed=0 en las 3**	Despliegue repetible y automatizado
`https://dominio/api/health`	**200 vía NLB → ALB → EC2**	Cadena completa con HTTPS

2 La arquitectura

Cómo están conectadas las piezas y por qué es un patrón conocido

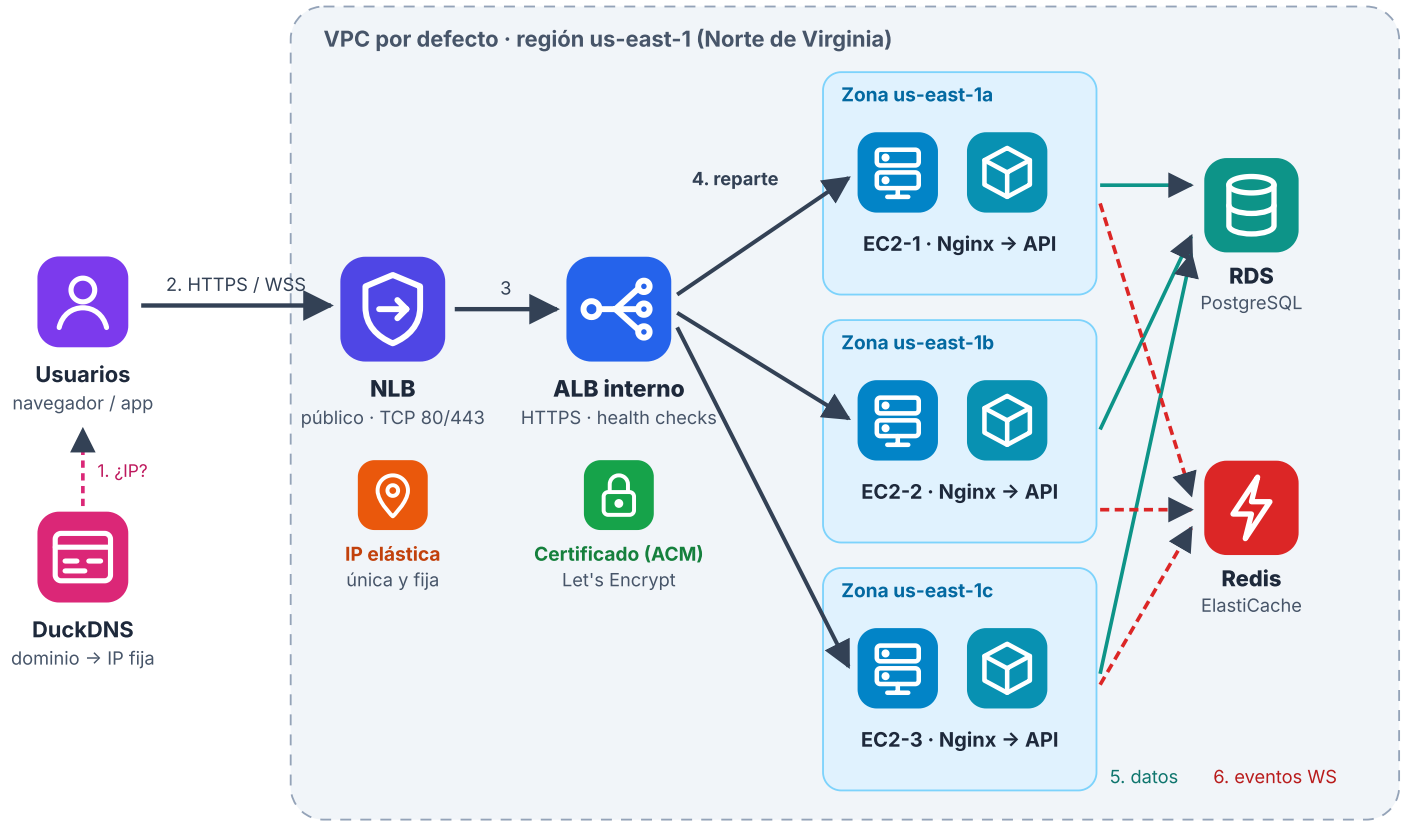


Figura 1. Flujo de una petición: el dominio resuelve a la IP elástica del NLB → el NLB pasa la conexión al ALB → el ALB descifra HTTPS y elige un servidor sano → Nginx entrega a la API (Docker) → la API usa RDS (datos) y Redis (eventos en tiempo real).

El viaje de una petición, paso a paso

1. **DNS**
   El navegador pregunta a DuckDNS la IP de `sysbarrestaurant.duckdns.org` y recibe la **IP elástica**.

2. **NLB (capa 4)**
   Recibe la conexión TCP en 80/443 y la reenvía tal cual (sin descifrar) al ALB. Conserva la IP real del cliente.

3. **ALB (capa 7)**
   Descifra HTTPS con el certificado de ACM, redirige HTTP→HTTPS, y elige una instancia **sana** según los health checks.

4 **Nginx en la EC2**
Recibe HTTP en el puerto 80 y lo pasa a la API en `127.0.0.1:3000`, incluyendo el "upgrade" a WebSocket.

5 **API en Docker**
NestJS procesa la petición; lee/escribe en RDS y publica eventos de Socket.IO en Redis.

6 **Redis**
Reparte cada evento a las otras instancias para que **todos** los clientes conectados lo reciban.

¿Cómo se llama esta arquitectura? ¿Es común?

Sí, es de las más usadas para backends en la nube. Combina varios patrones reconocidos:

Patrón	Qué significa aquí	Qué tan común es
Arquitectura de 3 capas	Entrada (balanceadores) → aplicación (EC2) → datos (RDS/Redis), cada capa con su propio firewall.	El modelo clásico de aplicaciones web; base del *AWS Well-Architected Framework*.
Escalado horizontal con servidores **stateless**	Varias copias idénticas de la API; ninguna guarda estado propio (sesión en JWT, datos en RDS, eventos en Redis).	Práctica estándar: permite agregar/quitar servidores sin afectar a los usuarios.
Alta disponibilidad Multi-AZ	Cada EC2 vive en una zona de disponibilidad distinta (1a, 1b, 1c).	Recomendación base de AWS para producción.
Pub/Sub con Redis para WebSockets	Adaptador de Socket.IO que sincroniza eventos entre instancias.	Solución documentada por Socket.IO para cualquier despliegue con más de un servidor.
NLB delante de ALB	El NLB aporta IP fija; el ALB, el balanceo inteligente HTTP.	Patrón documentado por AWS para cuando se necesita IP estática hacia un ALB.
Rolling deployment	Ansible actualiza un servidor a la vez (`serial: 1`).	Estrategia estándar para desplegar sin cortar el servicio.

Detalle técnico
Capa de entrada: NLB internet-facing (1 subnet + EIP, cross-zone activado) → target group tipo `alb` (TCP 80/443). ALB `internal` en 3 subnets, listener 80 → redirect 301, listener 443 (`ELBSecurityPolicy-TLS13-1-2-2021-06`) → target group HTTP:80 con health check `/api/health`, stickiness `lb_cookie` 1 día, idle timeout 3600 s, deregistration delay 60 s.

3 Servicios de AWS utilizados

Qué es cada uno y por qué está en la arquitectura

Dato importante

Todos los servicios están en la región **us-east-1 (Norte de Virginia)**, una de las más económicas. Los **Key Pairs** (llaves SSH) existen por región: una llave creada en Ohio no sirve en Virginia.

VPC (red privada)

La "colonia cerrada" donde viven todos los recursos. Usamos la **VPC por defecto** y sus subnets de las zonas a, b y c.

¿Por qué? Evita crear redes desde cero en un entorno de prueba.

EC2 (servidores)

3 computadoras virtuales `t3.micro` (2 vCPU, 1 GB RAM) con Ubuntu 24.04, Docker y Nginx.

¿Por qué? Es donde corre la API. Tamaño mínimo viable; `t3.nano` (0.5 GB) no alcanza para Docker + NestJS.

EBS (discos)

Un disco `gp3` de 16 GB por servidor.

¿Por qué? Las imágenes de Docker ocupan espacio; 8 GB era justo.

IP elástica (EIP)

Una dirección pública que no cambia.

¿Por qué? DuckDNS solo acepta IPs; necesitábamos una fija para el dominio.

Network Load Balancer

Balanceador de capa 4 (TCP) que sí admite IP elástica.

¿Por qué? Es la única forma de dar IP fija a un ALB sin servicios extra.

Application Load Balancer

Balanceador de capa 7 (HTTP/HTTPS) con health checks, redirecciones y sticky sessions.

¿Por qué? Era el requisito principal: repartir el tráfico entre las EC2.

Target Groups

Listas de destinos de cada balanceador (las 3 EC2 para el ALB; el ALB para el NLB).

¿Por qué? Así cada balanceador sabe a quién enviar y cómo revisar su salud.

ACM (certificados)

Guarda el certificado HTTPS que usa el ALB.

¿Por qué? ACM no emite certificados para duckdns.org, así que importamos uno de Let's Encrypt (gratis).

RDS PostgreSQL

Base de datos administrada `db.t4g.micro`, 20 GB, privada.

¿Por qué? Las 3 instancias comparten los mismos datos; AWS se encarga de parches y respaldos.

ElastiCache Redis

Redis administrado `cache.t4g.micro`.

¿Por qué? Sincroniza los eventos de Socket.IO entre servidores (sin él, la cocina no recibe órdenes creadas en otro servidor).

Security Groups

Firewalls por recurso: NLB, ALB, EC2, RDS y Redis.

¿Por qué? Cada capa solo acepta tráfico de la capa anterior (defensa en profundidad).

IAM Identity Center (SSO)

Inicio de sesión de la terminal con tu usuario de AWS.

¿Por qué? Credenciales temporales: más seguro que guardar claves permanentes.

4 Terraform y Ansible en pocas palabras

Las dos herramientas de Infraestructura como Código

Terraform — el arquitecto

Analogía
Es el **plano de la casa**: describes qué quieres (3 servidores, 1 balanceador, 1 base de datos) y Terraform lo construye en AWS. Si cambias el plano, solo modifica lo necesario.

- Archivos `.tf` en lenguaje HCL (**declarativo**: dices *qué*, no *cómo*).
- Guarda la "memoria" de lo creado en **terraform.tfstate**.
- Ciclo: `init` → `plan` (vista previa) → `apply` (crear/cambiar) → `destroy` (borrar).
- En este proyecto crea 29 recursos y escribe solo el inventario de Ansible.

Ansible — el instalador

Analogía
Es el **equipo que amuebla la casa**: entra a cada servidor por SSH y deja todo instalado y configurado igual en los tres (Docker, firewall, Nginx, la app).

- **Playbooks** `.yml` con tareas en orden.
- **Idempotente**: correrlo dos veces da el mismo resultado sin romper nada.
- **Agentless**: no instala nada en los servidores, solo usa SSH.
- Playbooks: `certificado.yml`, `duckdns.yml`, `deploy.yml`.

	Terraform	Ansible
Responde a	¿Qué infraestructura existe?	¿Cómo está configurado cada servidor?
Habla con	La API de AWS	Los servidores por SSH
Ejemplos aquí	EC2, ALB, NLB, RDS, Redis, firewalls, certificado	Swap, Docker, UFW, Fail2ban, Nginx, clonar repo, `docker compose up`
Se ejecuta desde	Raíz del repo	Carpeta `ansible/`

El puente entre los dos
Terraform genera automáticamente `ansible/inventory.ini` (IPs de las 3 EC2) y `ansible/group_vars/all/infra.yml` (endpoints de RDS/Redis, IP elástica). Así nunca se copian IPs a mano.

5 Lo que necesitas instalado y configurado

Herramientas, versiones mínimas, cuentas y archivos

Herramientas en tu computadora

Herramienta	Versión mínima	Usada en el proyecto	Instalación (macOS)	Para qué
AWS CLI	v2.x	v2	`brew install awscli`	Login SSO y consultas a AWS
Terraform	1.9	1.16.0	`brew install hashicorp/tap/terraform`	Crear la infraestructura
Ansible	ansible-core 2.15 + `community.general`	paquete `ansible` completo	`brew install ansible`	Configurar servidores y desplegar
Node.js	18	24	`brew install node`	Compilar la API y la prueba de WebSockets
Git + llave SSH de GitHub	2.x	—	incluido	Clonar el repo privado desde los servidores
openssl, dig, curl	—	—	incluidos en macOS	Certificado, DNS y pruebas

Funciona igual en Mac Intel o Apple Silicon (M1–M4) y en Linux. Providers de Terraform: `hashicorp/aws ~> 6.0`, `hashicorp/local ~> 2.5`, `hashicorp/time ~> 0.12` (se descargan con `terraform init`).

Cuenta de AWS

Con tarjeta registrada y un usuario con **AdministratorAccess** (Identity Center/SSO o usuario IAM dedicado). Nunca uses claves del usuario root.

Cuenta de DuckDNS

Gratis. Un subdominio (ej. `mi-app.duckdns.org`) y tu **token** (si el navegador traduce la página, aparece como “simbólico”).

Key Pair de EC2

Creado en **la misma región**, formato `.pem`, guardado en `~/.ssh/` con `chmod 400`.

Los archivos `.example` : tu configuración privada

Copiar, no renombrar

El repositorio no contiene IPs, contraseñas ni tokens. Trae **plantillas**. Debes **copiarlas** (no renombrarlas) y llenar tus datos. Las copias están en `.gitignore` y nunca se suben.

```
cp terraform.tfvars.example terraform.tfvars
cp ansible/vars.yml.example ansible/vars.yml
```

Archivo	Valores clave que debes poner
`terraform.tfvars`	`aws_region`, `aws_profile`, `domain_name`, `key_name`, `ssh_private_key_path`, `ssh_allowed_cidrs` (tu IPv4/32: `curl -4 ifconfig.me`), `db_password` (sin `@ / "` ni espacios), `health_check_path`
`ansible/vars.yml`	`domain_name`, `duckdns_subdomain`, `duckdns_token`, `acme_email`, `app_repo_url`, `app_repo_version`, `app_compose_file`, `app_extra_env` (ej. `JWT_SECRET`)
`inventory.ini.example`	**No se copia**: el real lo genera Terraform.

6 Lo que hicimos, paso a paso

La ruta completa desde el análisis hasta la verificación

1. **Análisis del proyecto original**

   Revisamos el repo `infra-sysbar`: Terraform y Ansible para un solo servidor, puertos de más abiertos y un error en el grupo del inventario.

2. **Decisión de arquitectura**

   Un ALB no puede tener IP elástica y DuckDNS no acepta CNAME. Opciones: dominio en Route 53 (A) o **NLB con IP elástica delante del ALB (B)**. Se eligió B para conservar el dominio; 3 instancias mínimas en us-east-1.

3. **Reescritura de Terraform**

   Red por zonas, AMI oficial de Ubuntu, 3 EC2, security groups por capas, ALB interno, NLB + EIP, certificado importado a ACM, Redis, outputs y generación del inventario de Ansible.

4. **Reescritura de Ansible**

   `certificado.yml` (Let's Encrypt por DNS-01 con DuckDNS), `duckdns.yml` (apunta el dominio), `deploy.yml` (servidores + despliegue rolling) y Nginx con soporte WebSocket.

5. **Credenciales y configuración**

   Copia de los `.example`, login con `aws configure sso`, Key Pair nuevo en Virginia, IP propia para SSH.

6. **Paso 1: certificado HTTPS**

   `ansible-playbook certificado.yml` → certificado válido hasta el 29-ene-2027 en `certs/`.

7. **Paso 2: infraestructura**

   `terraform init / plan / apply` → 29 recursos en ~6 minutos.

8. **Paso 3: dominio**

   `ansible-playbook duckdns.yml` → el dominio apunta a la IP elástica.

9. **Paso 4: servidores**

   `ansible-playbook deploy.yml` → swap, Docker, firewall, Fail2ban, Nginx en las 3 EC2.

10

Adaptación de la API (SysRanchito)

Adaptador de Redis para Socket.IO, endpoint `/api/health`, `trust proxy`, `docker-compose.prod.yml`, `.dockerignore` y workflow viejo en modo manual.

11

Paso 5: despliegue de la API

Rolling deploy servidor por servidor. Se corrigió un bloqueo de SSH durante el `docker build` (async/poll + keep-alive).

12

Paso 6: verificación

Balanceo entre 3 instancias, Redis conectado en las 3 y prueba de WebSockets 9/9.

13

Orden y documentación

14 commits temáticos, plantillas `.example` sin datos personales, README completo y esta guía.

7 Cómo funciona el ALB

Reparto, salud, afinidad y capacidad

En palabras simples

El ALB es el **recepcionista**: recibe a cada visitante, revisa cada 15 segundos qué "cajeros" (servidores) están atendiendo bien, y manda a cada persona con uno disponible. Si un cajero deja de responder, deja de mandarle gente.

Listeners

Puerto 80: redirige a HTTPS (301).
Puerto 443: descifra TLS y envía al target group.

Health checks

`GET /api/health` cada **15 s**. Sano tras **2** respuestas 200; fuera tras **3** fallas (~45 s).

Sticky sessions

Cookie `AWSALB` (1 día): el mismo usuario va al mismo servidor. Necesaria para el long-polling de Socket.IO.

WebSockets

Idle timeout de **3600 s** (igual en NLB y Nginx) para no cortar conexiones en tiempo real.

Drenado

Al sacar un servidor, espera **60 s** para que terminen las conexiones activas.

TLS

TLS 1.2/1.3 con la política `ELBSecurityPolicy-TLS13-1-2-2021-06`.

¿Por qué en el navegador siempre veo el mismo servidor?

Por la cookie de afinidad (sticky session). Con `curl`, que no guarda cookies, se ven las 3 instancias alternarse. Es comportamiento esperado: usuarios distintos sí se reparten.

Mínimo, máximo y crecimiento automático

Una aclaración importante: **el ALB reparte tráfico, pero no crea servidores**. Quien agrega o quita instancias es un **Auto Scaling Group (ASG)**. Hoy el proyecto tiene una cantidad **fija**:

Concepto	Configuración actual	Con Auto Scaling (mejora recomendada)
Instancias mínimas	**3** (fijas)	**2** (nunca menos, para alta disponibilidad)
Instancias deseadas	3	3 en horario normal
Instancias máximas	**3** (o hasta **10** cambiando `instance_count` a mano + `terraform apply` + `deploy.yml`)	Por ejemplo **10** (límite que tú defines; el tope real lo marcan las cuotas de tu cuenta de AWS)
Cuándo crece	Nunca solo	Si el CPU promedio pasa de ~60 % o hay muchas peticiones por servidor (**target tracking**)

Cuándo se reduce	Nunca solo	Cuando el tráfico baja, regresa al mínimo (2) para ahorrar
Si un servidor muere	El ALB deja de enviarle tráfico; hay que recrearlo con Terraform	El ASG lo reemplaza automáticamente

Detalle técnico

El ALB en sí escala de forma transparente: AWS agrega capacidad (nodos) según el tráfico y lo cobra en LCU. No hay que configurar nada para ello. Los límites de instancias los define tu ASG y las cuotas de servicio de EC2 de la cuenta.

8 El NLB: para qué sirve y por qué lo colocamos

La pieza que da la IP fija

En palabras simples

El NLB es la **puerta principal con dirección fija**. DuckDNS necesita una dirección fija para tu dominio, pero el ALB cambia de dirección cuando quiere. Entonces ponemos una puerta con dirección fija (NLB) que pasa a todos los visitantes directo al recepcionista (ALB).

El problema

- Un ALB **no puede tener IP elástica**: AWS le asigna IPs que cambian.
- DuckDNS solo acepta registros **A** (IP), no **CNAME** (nombres).
- Apuntar el dominio a la IP actual del ALB dejaría el sitio caído cuando esa IP cambie.

La solución

- Un **NLB** con **1 IP elástica** (una sola subnet → una sola IP).
- Reenvía **TCP** 80 y 443 al ALB sin descifrar nada (**TLS passthrough**).
- Conserva la **IP real del cliente**, que llega a la API vía `X-Forwarded-For`.
- Idle timeout TCP de 3600 s para WebSockets.

	NLB (capa 4)	ALB (capa 7)
Entiende	Conexiones TCP/UDP	HTTP, HTTPS, rutas, cookies, WebSockets
IP fija / elástica	**Sí**	**No**
Termina HTTPS	Puede, pero aquí no	**Sí (con ACM)**
Health checks	Revisa al ALB	Revisa a cada EC2 (`/api/health`)
Rol aquí	Dirección fija para DuckDNS	Reparto inteligente entre las EC2

Compromiso aceptado

El NLB vive en **una sola zona** (us-east-1a) para tener **una sola IP**. Si esa zona entera de AWS fallara, el sitio caería aunque las otras EC2 estén bien. Con un dominio propio en Route 53 se podría eliminar el NLB y apuntar directo al ALB en 3 zonas.

Detalle técnico

`aws_lb.nlb` con `subnet_mapping` + `allocation_id` de la EIP, `enable_cross_zone_load_balancing = true`, security group propio. Target groups `target_type = "alb"` (TCP 80 y 443) con health check HTTP/HTTPS. Al destruir, un `time_sleep` de 60 s evita el error `ResourceInUse` en los listeners del ALB.

9 Costos

Cuánto cuesta cada pieza, por minuto, por día y por mes

En palabras simples

En la nube se paga como la luz: **por el tiempo que las cosas están encendidas**, se usen o no. Un balanceador sin visitantes cobra igual que uno con pocos visitantes. Por eso, en pruebas, conviene **apagar todo** cuando no se usa.



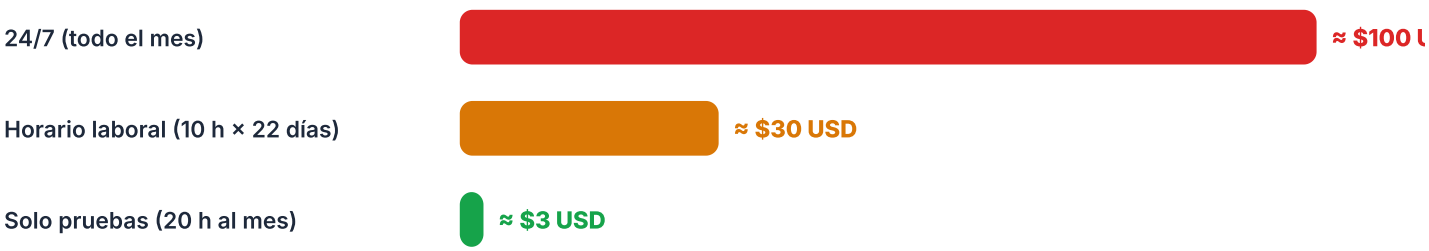
Figura 2. Mapa de costos mensuales (24/7, us-east-1, precios de lista bajo demanda). El grosor de cada rama es proporcional a su costo.

Servicio	Precio de lista	Por hora	Por minuto	Por día	Por mes (730 h)	Cobro
EC2 t3.micro ×3	$0.0104 c/u	$0.0312	$0.00052	$0.75	**$22.78**	Por segundo (mín. 60 s)
Discos gp3 16 GB ×3	$0.08 / GB-mes	$0.0053	$0.00009	$0.13	**$3.84**	Prorrateado
ALB	$0.0225 + LCU	$0.0225	$0.00038	$0.54	**$16.43**	Por hora o fracción
NLB	$0.0225 + NLCU	$0.0225	$0.00038	$0.54	**$16.43**	Por hora o fracción

RDS db.t4g.micro	$0.016	$0.0160	$0.00027	$0.38	**$11.68**	Por segundo (mín. 10 min)
Disco RDS gp2 20 GB	$0.115 / GB-mes	$0.0032	$0.00005	$0.08	**$2.30**	Prorrateado
ElastiCache cache.t4g.micro	$0.016	$0.0160	$0.00027	$0.38	**$11.68**	Por hora o fracción
IPv4 públicas ×4	$0.005 c/u	$0.0200	$0.00033	$0.48	**$14.60**	Por hora
TOTAL		**$0.1366**	**$0.0023**	**$3.28**	**≈ $99.74**	

Las 4 IPs públicas: 1 elástica (NLB) + 3 de las EC2 (para salida a internet y SSH). El ALB es interno, así que no consume IPs públicas. No incluye LCU/NLCU ni transferencia de datos de salida (con tráfico bajo suman unos pocos dólares; los primeros 100 GB/mes de salida a internet son gratis). Si tu cuenta tiene créditos o capa gratuita, el costo real puede ser menor.

¿Encendido 24/7 o solo cuando se usa?



Por qué producción va 24/7

- Los usuarios (meseros, caja, cocina) usan el sistema a cualquier hora; si está apagado, no hay servicio.
- Los WebSockets requieren servidores escuchando todo el tiempo.
- `terraform destroy` **borra la base de datos**: no sirve para "pausar" producción.
- Encender todo de nuevo toma ~25 min y cambia la IP elástica.

Cómo ahorrar

- **Pruebas:** `terraform destroy` al terminar (~$0.14 por hora de uso).
- **Savings Plans / Reservas** de 1 año: 30–40 % menos en EC2, RDS y Redis.
- Instancias **t4g** (ARM/Graviton): ~20 % más baratas.
- Dominio propio en Route 53: elimina el NLB (−$16) y su IP.
- Auto Scaling: bajar a 2 instancias en horas tranquilas.

10 ¿Solo funciona con SysRanchito / NestJS?

Respuesta corta: no. Cómo reutilizarlo con cualquier backend

Respuesta corta: NO
La infraestructura es **agnóstica a la tecnología**. Funciona con cualquier aplicación que cumpla un {kw("contrato")} de 4 puntos. SysRanchito/NestJS es solo la primera aplicación que lo cumple.

El contrato que debe cumplir cualquier aplicación

1. Contenedor Docker

Un `Dockerfile` y un `docker-compose.prod.yml` que publique en `127.0.0.1:PUERTO`.

2. Health check

Una ruta que responda **200** (ej. `/api/health`, `/up`, `/actuator/health`).

3. Config por variables

Lee BD, Redis y secretos desde variables de entorno (`.env` generado por Ansible).

4. Stateless

Nada importante en memoria o disco local: sesiones en JWT/Redis, archivos en S3.

Qué cambiar según el caso

Escenario	Qué cambiar	Por qué
Otra API en NestJS	`vars.yml`: `app_repo_url`, `app_repo_version`, `app_compose_file`, `app_service`, `app_extra_env`. Agregar `/api/health` y, si usa Socket.IO, el adaptador de Redis.	Mismo stack: solo cambia de dónde viene el código y sus variables.
Node.js (Express, Fastify)	Igual que Nest. Socket.IO → `@socket.io/redis-adapter`. Ajustar `health_check_path` y nombres de variables en `app.env.j2`.	El contrato es el mismo; solo cambian rutas y nombres.
Laravel (PHP)	Contenedor que sirva HTTP en un puerto (PHP-FPM + Nginx interno, Octane o FrankenPHP); `app_port`; health `/up`; normalmente **MySQL** → `engine = "mysql"` en `rds.tf`; `SESSION_DRIVER=redis`, `CACHE_STORE=redis`; WebSockets con **Laravel Reverb** (escala con Redis); colas y `schedule:run` en **una sola** instancia.	PHP necesita sesiones compartidas y cron no duplicado entre servidores.
Java (Spring Boot)	`app_port: 8080`; health `/actuator/health`; `DB_*` → `SPRING_DATASOURCE_*` en `app.env.j2`; WebSockets STOMP con **broker relay** externo o Spring Session + Redis; más RAM (`t3.small` / `t3.medium`).	La JVM consume más memoria; STOMP necesita un broker compartido.
Python (Django / FastAPI)	`app_port: 8000`; health propio; Django Channels con `channels_redis` o FastAPI + Redis pub/sub.	Mismo principio: los eventos en tiempo real pasan por Redis.

Go, .NET, Ruby...	Puerto, health y variables. .NET SignalR → *Redis backplane*.	Cualquier lenguaje que corra en Docker sirve.
Sin WebSockets	`enable_redis = false` y `enable_sticky_sessions = false`.	Ahorra ~$12/mes y reparte cada petición.
Con dominio propio	Route 53 + certificado ACM gratuito; se puede eliminar el NLB y `certificado.yml`.	Más simple, renovación automática y Multi-AZ real.

Archivos que normalmente se tocan
`ansible/vars.yml` (repo, rama, puerto, variables), `ansible/app.env.j2` (nombres de variables), `terraform.tfvars` (`health_check_path`, tamaños, `enable_redis`), `rds.tf` (motor de BD) y, rara vez, `ansible/nginx.conf.j2` (tamaño de subidas, timeouts).

11 Qué hacer en caso de fallas

Diagnóstico ordenado y errores conocidos

Diagnóstico de afuera hacia adentro

1. **¿El dominio apunta a la IP correcta?**
   `dig +short tu-dominio.duckdns.org` debe mostrar el `public_ip` de Terraform. Si no: `ansible-playbook duckdns.yml`.

2. **¿El NLB y el ALB responden?**
   `curl -I http://tu-dominio` → 301. Si no responde: revisa en la consola EC2 → Load Balancers.

3. **¿Hay instancias sanas?**
   `aws elbv2 describe-target-health …` (ver README). Si todas están *unhealthy*, el problema está en la app o en Nginx.

4. **¿Nginx y el contenedor están arriba?**
   `ansible webservers -b -m shell -a "systemctl is-active nginx; docker ps"`

5. **¿Qué dice la aplicación?**
   `docker logs --tail 50 CONTENEDOR`: errores de BD, Redis o variables de entorno.

6. **¿Llegan a RDS y Redis?**
   Revisa `DB_HOST` / `REDIS_URL` en el `.env` y que los security groups permitan 5432/6379 desde las EC2.

Errores conocidos (todos ocurrieron durante el proyecto)

Síntoma	Causa	Solución
`No valid credential sources found`	Terraform no tiene credenciales de AWS	Revisa `aws_profile` en `terraform.tfvars` y corre `aws sso login --profile TU_PERFIL`
`Token has expired and refresh failed`	La sesión SSO venció (dura horas)	`aws sso login --profile TU_PERFIL`
`Error: No configuration files`	Terraform se ejecutó dentro de `ansible/`	`cd ..` a la raíz del repositorio
`InvalidKeyPair.NotFound`	El Key Pair no existe en esa región	Créalo en la región de `aws_region`

`Error de RDS con la contraseña`	Caracteres no permitidos	Usa una contraseña sin `@ / "` ni espacios
`Configura duckdns_token…`	`vars.yml` con valores de ejemplo	Pon tu token real (en DuckDNS: "token" o "simbólico")
`UNREACHABLE / timed out en Ansible`	Tu IP pública cambió	`curl -4 ifconfig.me` → actualiza `ssh_allowed_cidrs` → `terraform apply`
`curl ifconfig.me devuelve 2806:…`	Es tu IPv6	Usa `curl -4 ifconfig.me`
`Ansible se queda detenido mucho tiempo`	Servidor ocupado (1 GB de RAM) durante el build	Espera; si pasan >15 min sin avance, `Ctrl+C` y repite (es seguro)
`El sitio responde 502 / 503`	La API no corre o falla el health check	`ansible webservers -b -m shell -a "docker ps -a; docker logs --tail 30 CONTENEDOR"`
`destroy: ResourceInUse en un listener`	AWS aún no suelta el ALB del NLB	Espera 2–5 min y repite `terraform destroy`
`VS Code marca variables "no esperadas"`	El editor guardó una versión vieja del archivo	Cierra sin guardar; ante "archivo cambió en disco", elige **Revert**
`WebSockets llegan solo a algunos clientes`	Una instancia sin conexión a Redis	Revisa logs: debe decir "Socket.IO conectado a Redis"

¿Qué pasa si...?

Escenario	Comportamiento	Qué hacer
Se cae 1 instancia	En ~45 s el ALB la saca; las otras 2 atienden	Revisar logs; `deploy.yml --limit` o recrearla con Terraform
Se cae la zona us-east-1a	Cae el NLB (única IP) → sitio fuera	Esperar a AWS; a futuro, Route 53 + ALB Multi-AZ
Se cae Redis	La API sigue funcionando; los eventos en tiempo real solo llegan dentro del mismo servidor	Revisar ElastiCache; reiniciar contenedores al volver
Se cae RDS	La API responde con errores de base de datos	Revisar RDS en la consola; para producción: Multi-AZ y respaldos
Un despliegue sale mal	Rolling: el primer servidor falla y Ansible se detiene; los otros 2 siguen con la versión anterior	Corregir y redesplegar, o volver a la versión anterior con `app_repo_version`
Vence el certificado	Los navegadores muestran "no seguro"	`certificado.yml` + `terraform apply` antes del vencimiento

12 Operación del día a día

Los comandos que más vas a usar

Quiero…	Comando
Iniciar sesión en AWS	`aws sso login --profile TU_PERFIL`
Desplegar una nueva versión de la API	`git push` en la rama configurada → `ansible-playbook deploy.yml --tags app`
Forzar reconstrucción	`ansible-playbook deploy.yml --tags app -e force_deploy=true`
Actualizar un solo servidor	`ansible-playbook deploy.yml --limit sysranchito-dev-ec2-1`
Ver logs de los 3 servidores	`ansible webservers -b -m shell -a "docker logs --tail 50 CONTENEDOR"`
Probar balanceo	`for i in 1 2 3 4 5 6; do curl -s https://DOMINIO/api/health; echo; done`
Probar WebSockets	`cd scripts && npm install && node test-websockets.mjs https://DOMINIO`
Renovar certificado (cada ~60 días)	`ansible-playbook certificado.yml` → `terraform apply`
Apagar todo (pruebas)	`terraform destroy` (borra también la BD)
Encender de nuevo	`terraform apply` → `ansible-playbook duckdns.yml` → `ansible-playbook deploy.yml`

Buena práctica

Antes de cada `terraform apply`, lee el resumen del `plan`: si dice **“to destroy”** con un número mayor que 0 y no lo esperabas, **no confirmes**.

13 Seguridad

Cada capa solo acepta tráfico de la capa anterior

Recurso	Acepta tráfico de	Puertos
NLB	Internet	80, 443
ALB (interno, sin IP pública)	El NLB (solo alcanzable dentro de la VPC)	80, 443
EC2	Solo el ALB (HTTP) y tu IP (SSH)	80, 22
API (Docker)	Solo Nginx en la misma máquina (`127.0.0.1`)	3000
RDS	Solo las EC2	5432
Redis	Solo las EC2	6379

Cifrado

HTTPS con TLS 1.2/1.3 y redirección automática de HTTP a HTTPS. Conexión a RDS con SSL.

Servidores endurecidos

UFW (solo 22 y 80), Fail2ban contra ataques de SSH, IMDSv2 obligatorio.

Secretos fuera de git

`.tfvars` , `vars.yml` , `certs/` , inventario y `tfstate` en `.gitignore` . Solo se suben plantillas `.example` .

Cuidado con terraform.tfstate

Contiene la llave privada del certificado y la contraseña de la BD. No lo compartas ni lo subas. Para equipos, guárdalo en un backend S3 cifrado con bloqueo.

14 Mejoras recomendadas

Ordenadas por impacto

#	Mejora	Beneficio
1	**Auto Scaling Group** + Launch Template	Crece y se reduce con el tráfico; reemplaza servidores caídos solo.
2	Imagen Docker construida en CI y guardada en **ECR**	Despliegues de segundos en lugar de ~5 min por servidor.
3	Dominio propio en Route 53 + ACM	Elimina el NLB, el riesgo de una sola zona y la renovación manual del certificado.
4	Migraciones de TypeORM (quitar `synchronize: true`)	Cambios de esquema controlados antes de tener datos reales.
5	Estado de Terraform en S3 + bloqueo	Trabajo en equipo seguro y respaldo del estado.
6	SSM Session Manager en lugar de SSH	Cerrar el puerto 22 por completo.
7	RDS Multi-AZ y respaldos automáticos	La base de datos sobrevive a la caída de una zona.
8	Monitoreo con CloudWatch (alarmas de CPU, 5xx, hosts sanos)	Enterarte de un problema antes que tus usuarios.

15 Glosario de palabras clave

Las llaves para entender esta arquitectura

ALB
Application Load Balancer. Balanceador HTTP/HTTPS (capa 7) que reparte entre servidores y revisa su salud.

NLB
Network Load Balancer. Balanceador TCP (capa 4); admite IP elástica.

IP elástica (EIP)
IP pública fija de AWS que se asigna a un recurso.

Target Group
Lista de destinos de un balanceador y la regla para revisar su salud.

Health check
Revisión periódica de una ruta (ej. /api/health) para saber si un servidor está sano.

Sticky session
Afinidad: un usuario se queda con el mismo servidor gracias a una cookie.

Idle timeout
Segundos que una conexión inactiva se mantiene abierta.

Zona de disponibilidad (AZ)
Centro de datos independiente dentro de una región (us-east-1a, 1b, 1c).

Alta disponibilidad
Diseño que sigue funcionando aunque falle una parte.

Escalado horizontal
Agregar más servidores iguales (en vez de uno más grande, que es el vertical).

Auto Scaling Group
Servicio que ajusta automáticamente el número de servidores.

Stateless
La aplicación no guarda estado propio; cualquier servidor puede atender cualquier petición.

IaC
Infraestructura como Código: la infraestructura se describe en archivos versionados.

Terraform state
Archivo donde Terraform recuerda lo que creó. Sensible.

Playbook
Lista ordenada de tareas de Ansible.

Idempotente
Ejecutarlo varias veces produce el mismo resultado.

Rolling deploy
Actualizar servidores uno por uno sin cortar el servicio.

WebSocket
Conexión permanente para comunicación en tiempo real (órdenes, avisos).

Pub/Sub (Redis)
Mecanismo de publicar/suscribir que reparte eventos entre servidores.

TLS / HTTPS
Cifrado de la comunicación; el candado del navegador.

ACM
AWS Certificate Manager: guarda certificados para los balanceadores.

DNS-01
Forma de demostrar que el dominio es tuyo creando un registro TXT.

Security Group
Firewall de AWS asociado a cada recurso.

SSO
Inicio de sesión único (IAM Identity Center) con credenciales temporales.

LCU / NLCU
Unidades de capacidad con las que AWS cobra el uso de ALB / NLB.

Swap
Memoria de respaldo en disco; evita que el servidor se quede sin RAM al compilar.

Fuentes de precios

- AWS — Elastic Load Balancing pricing (ALB y NLB: $0.0225/h + LCU $0.008 / NLCU $0.006, us-east-1).
- AWS News Blog — Public IPv4 Address Charge ($0.005 por IP por hora, desde feb-2024).
- AWS — Amazon EC2 T3 instances (t3.micro: $0.0104/h, Linux, us-east-1).
- AWS — Amazon RDS for PostgreSQL pricing (db.t4g.micro Single-AZ: $0.016/h).
- AWS — Amazon ElastiCache pricing (cache.t4g.micro Redis OSS: $0.016/h).

Precios de lista bajo demanda en us-east-1, verificados en noviembre de 2026; pueden cambiar. Confírmalos en la calculadora oficial de AWS (calculator.aws).